HYACINTH AI

Security & Privacy Whitepaper

Mnemne — local-first document intelligence for macOS

How Mnemne keeps confidential and privileged documents private: everything is processed and stored on the user's own Mac, and nothing about your documents leaves the device in normal use.

Product	Mnemne (macOS desktop application)
Vendor	Hyacinth AI
Document	Security & Privacy Whitepaper, v1.0
Audience	Customer IT, information-security & vendor-risk reviewers
Published	June 2026
Contact	security@hyacinth.ai

1. Executive summary

Mnemne is a **local-first** document search and AI assistant. It indexes documents on the user's own Mac and answers questions about them using an **on-device** large language model. For organizations that handle confidential or privileged material — law firms in particular — the central guarantee is simple:

> **Your documents, their text, and the AI's analysis of them never leave the user's machine in the course of normal use.** There is no Hyacinth-operated cloud that receives, stores, or processes customer documents, and no third-party AI service that sees document content.

The only network activity in normal use is checking for application updates, a one-time download of the local AI model, and license validation. Optional, user-initiated features (web search, Google Drive import) are the only paths by which any data leaves the device, and each is described explicitly below.

2. Data-handling model

2.1 Where data lives

All user data is stored locally under the user's home directory:

Data	Location	Notes
Search index (document text + vector embeddings)	`~/.mnemne/data/`	Derived from the user's files
Application logs	`~/.mnemne/logs/`	Diagnostic only; secrets redacted (§10)
On-device AI models	`~/.mnemne/models/`	Downloaded once from Hugging Face
OAuth tokens (if a connector is used)	macOS **Keychain**	Never in plaintext files (§6)
The user's source documents	Wherever the user already keeps them	Indexed in place; local files are not copied

2.2 What leaves the device — and what never does

Never leaves the device in normal use: document contents, file names or paths, the text Mnemne extracts, vector embeddings, the questions a user asks, the AI's answers, or the search index.

Leaves the device only for these specific, disclosed purposes:

- **Application updates** — a signed update manifest is fetched from GitHub (§9). No user data is transmitted.
- **Model download** — the AI model is downloaded once from Hugging Face. No user data is transmitted.
- **License validation** — a license key is validated against the licensing provider. No document data is transmitted.
- **Web search (opt-in)** — if the user explicitly runs a web search, the query text (not document content) is sent to the configured search provider. Disabled by default.
- **Google Drive import (opt-in)** — files the user explicitly selects are fetched *from* Google to be indexed locally (§7). This is inbound; nothing about the local corpus is sent to Google.

2.3 On-device AI — no third-party model sees content

The language and embedding models run **entirely on the user's Mac** via a bundled local inference engine, listening only on loopback. Document content is **never** sent to OpenAI, Anthropic, Google, or any hosted model API, and there is **no training on customer data** — the model is a fixed, read-only artifact.

3. Architecture & trust boundaries

Mnemne is a single macOS application bundle containing four cooperating components, all running locally as the user and bound to loopback (`127.0.0.1`) only:

Component	Role	Listens on
Native shell (Rust)	Window, lifecycle, OS integration, updates	—
Application server	App / API logic	`127.0.0.1:3002`
Document parser (sidecar)	Text extraction	`127.0.0.1:9721`
On-device inference	LLM + embeddings	`127.0.0.1:8920/8921`

None of these are reachable from the network. The primary trust boundary is the **device itself**: a user who can run code as the logged-in account is inside the boundary (true of any desktop application). Mnemne's defenses focus on realistic external threats — malicious web pages, malicious documents, and tampering with the update or OAuth channels (§8).

4. Network egress inventory

For customers who restrict outbound traffic, this is the complete set of destinations Mnemne may contact. All external connections use **HTTPS/TLS**. The product's core function (local search + on-device AI) works fully offline.

Destination	Purpose	When
GitHub (HyacinthAI/pub-mnemne)	Update manifest + signed update download	Automatic, periodic
Hugging Face	One-time AI model download	First run / model setup
License provider	License key validation	Activation + periodic
Google (accounts / drive / oauth2 / apis)	Drive OAuth + file import	Only if the user connects Drive
Configured web-search provider	Web search queries	Only if the user runs a web search
Connectivity check (1.1.1.1)	Online/offline detection	Periodic, no payload

5. Data at rest & in transit

- **At rest:** the index and logs live under `~/.mnemne/`. Mnemne relies on **macOS FileVault** for encryption at rest; with FileVault enabled (recommended and typically mandated by customer policy) they are encrypted with the volume. App-level index encryption is on the roadmap (§11).
- **In transit:** inter-component traffic is loopback-only and never touches the network; all external traffic is HTTPS/TLS.
- **Secrets** (OAuth tokens) are not stored under `~/.mnemne/`; they live in the macOS Keychain (§6).

6. Authentication & secrets

- Mnemne has **no user accounts** and operates **no authentication server**. There is no Hyacinth-operated backend holding customer credentials or data.
- Optional cloud connectors use the OAuth **PKCE** flow with single-use state. Access and refresh tokens are stored in the macOS **Keychain**, never in plaintext on disk; OAuth scope is read-only.
- OAuth authorization codes and tokens are **redacted from application logs**.

7. Cloud connectors (Google Drive)

The Google Drive connector is **opt-in** and user-driven, designed to minimize data exposure:

- The user explicitly authenticates and **picks** the specific files or folder to index. Mnemne never enumerates the whole Drive on its own.
- For each selected file, Mnemne **streams the bytes to a temporary file, extracts text, and deletes the bytes.** Only extracted text + embeddings persist locally; the original file is not retained on the device.

- Indexed cloud documents link back to the **cloud copy**, not a local one.
- Disconnecting an account or removing a selection **purges** the corresponding indexed content from the local store.

8. Application integrity & local hardening

- Distributed as a **Developer ID-signed**, **Apple-notarized**, and stapled application, running under the macOS **Hardened Runtime**. Every executable, library, and native module inside the bundle is signed before notarization.
- Because Mnemne runs a local HTTP API, two same-machine browser attacks are explicitly defended: **DNS rebinding** (the API rejects non-loopback `Host` headers) and **cross-site request forgery** (state-changing requests with a non-loopback `Origin` are rejected).
- Outbound fetches for the optional web-search feature are **SSRF-guarded** — requests to loopback, private, link-local, or cloud-metadata addresses are rejected.
- The document parser enforces **size and "zip-bomb" limits** before any file is parsed, and Office-format XML is parsed with external-entity resolution disabled (no XXE).
- Custom-scheme (`mnemne://`) deep links are **source-gated** so a web page cannot forge a callback. Rendered content is escaped (no raw HTML), and a **Content-Security-Policy** restricts script/object/frame sources in the webview.

9. Auto-update security

- Update payloads are signed with an **Ed25519** key; the application embeds the corresponding public key and verifies every update's signature before applying it. An unsigned or tampered payload is refused.
- Updates are **manual** — the user is shown an "update available" banner and chooses to install. There is no silent or forced auto-update.
- The signing private key is held offline by the vendor, stored with restricted permissions and backed up in a password manager; only the public key ships in the application.

10. Logging & data minimization

- Logs are local-only (`~/.mnemne/logs/`) and exist for diagnostics.
- **Secrets are redacted** — OAuth codes, PKCE state, and callback parameters are stripped before writing.
- **No analytics, telemetry, or usage tracking of any kind** is present — verified by source audit (no analytics/telemetry SDKs in the codebase).

11. Known limitations & hardening roadmap

Disclosed proactively:

1. **At-rest index encryption** beyond FileVault — under evaluation.
2. **Hardened Runtime entitlements** — reviewing whether library-validation can be tightened now that all bundled libraries are signed under one Team ID.
3. **Independent third-party assessment** — no external SOC 2 or penetration test has been commissioned yet; an internal security program is the current control, and an external assessment is on the roadmap.

12. Incident response & contact

Security contact: **security@hyacinth.ai**. Given the local-first architecture, a server-side breach of customer documents is not an applicable failure mode — there is no server holding them. Because all data is local, deletion is fully in the user's control: removing `~/.mnemne/` and disconnecting any connector removes all Mnemne-held data; the vendor holds no copy.
